

TOPIC 3 : DIVIDE AND CONQUER

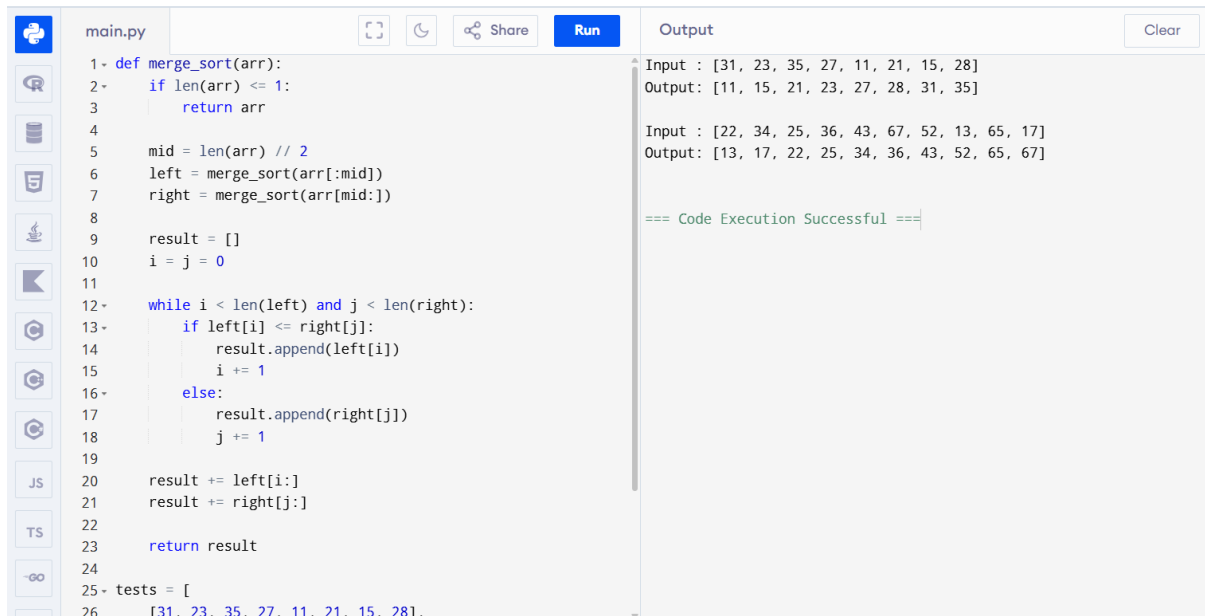
1.Maximum and Minimum in Array

main.py	Output
<pre>1- def find_min_max(arr): 2 minimum = arr[0] 3 maximum = arr[0] 4 5- for i in range(1, len(arr)): 6- if arr[i] < minimum: 7 minimum = arr[i] 8- if arr[i] > maximum: 9 maximum = arr[i] 10 11 return minimum, maximum 12 13- tests = [14 [5, 7, 3, 4, 9, 12, 6, 2], 15 [1, 3, 5, 7, 9, 11, 13, 15, 17], 16 [22, 34, 35, 36, 43, 67, 12, 13, 15, 17] 17] 18 19- for arr in tests: 20 minimum, maximum = find_min_max(arr) 21 print("Input:", arr) 22 print("Min =", minimum, "Max =", maximum) 23 print()</pre>	Input: [5, 7, 3, 4, 9, 12, 6, 2] Min = 2 Max = 12 Input: [1, 3, 5, 7, 9, 11, 13, 15, 17] Min = 1 Max = 17 Input: [22, 34, 35, 36, 43, 67, 12, 13, 15, 17] Min = 12 Max = 67 === Code Execution Successful ===

2. Min-Max in Sorted Array

main.py	Output
<pre>1- def find_min_max(arr): 2 minimum = arr[0] 3 maximum = arr[0] 4 5- for i in range(1, len(arr)): 6- if arr[i] < minimum: 7 minimum = arr[i] 8- if arr[i] > maximum: 9 maximum = arr[i] 10 11 return minimum, maximum 12 13- tests = [14 [2, 4, 6, 8, 10, 12, 14, 18], 15 [11, 13, 15, 17, 19, 21, 23, 35, 37], 16 [22, 34, 35, 36, 43, 67, 12, 13, 15, 17] 17] 18 19- for arr in tests: 20 minimum, maximum = find_min_max(arr) 21 print("Input:", arr) 22 print("Min =", minimum, "Max =", maximum) 23 print()</pre>	Input: [2, 4, 6, 8, 10, 12, 14, 18] Min = 2 Max = 18 Input: [11, 13, 15, 17, 19, 21, 23, 35, 37] Min = 11 Max = 37 Input: [22, 34, 35, 36, 43, 67, 12, 13, 15, 17] Min = 12 Max = 67 === Code Execution Successful ===

3. Merge Sort



```
1- def merge_sort(arr):
2-     if len(arr) <= 1:
3-         return arr
4-
5-     mid = len(arr) // 2
6-     left = merge_sort(arr[:mid])
7-     right = merge_sort(arr[mid:])
8-
9-     result = []
10-    i = j = 0
11-
12-    while i < len(left) and j < len(right):
13-        if left[i] <= right[j]:
14-            result.append(left[i])
15-            i += 1
16-        else:
17-            result.append(right[j])
18-            j += 1
19-
20-    result += left[i:]
21-    result += right[j:]
22-
23-    return result
24-
25- tests = [
26-     [31, 23, 35, 27, 11, 21, 15, 28],
```

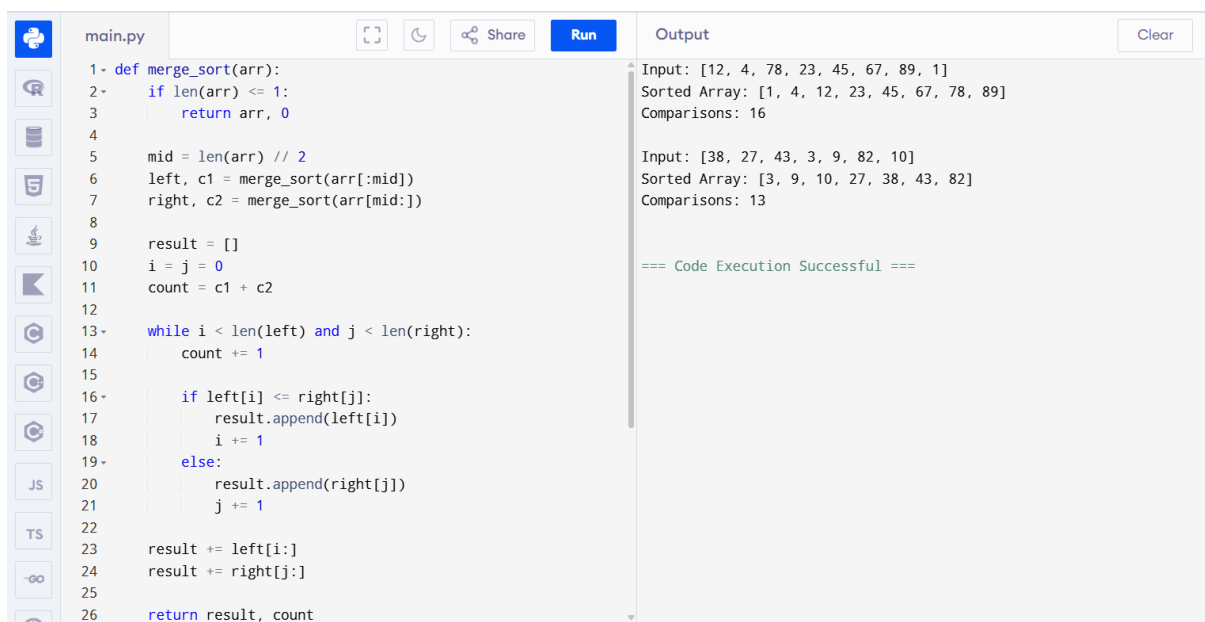
Output

Input : [31, 23, 35, 27, 11, 21, 15, 28]
Output: [11, 15, 21, 23, 27, 28, 31, 35]

Input : [22, 34, 25, 36, 43, 67, 52, 13, 65, 17]
Output: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]

=== Code Execution Successful ===

4. Merge Sort with Comparison Count



```
1- def merge_sort(arr):
2-     if len(arr) <= 1:
3-         return arr, 0
4-
5-     mid = len(arr) // 2
6-     left, c1 = merge_sort(arr[:mid])
7-     right, c2 = merge_sort(arr[mid:])
8-
9-     result = []
10-    i = j = 0
11-    count = c1 + c2
12-
13-    while i < len(left) and j < len(right):
14-        count += 1
15-
16-        if left[i] <= right[j]:
17-            result.append(left[i])
18-            i += 1
19-        else:
20-            result.append(right[j])
21-            j += 1
22-
23-    result += left[i:]
24-    result += right[j:]
25-
26-    return result, count
```

Output

Input: [12, 4, 78, 23, 45, 67, 89, 1]
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

Input: [38, 27, 43, 3, 9, 82, 10]
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
Comparisons: 13

=== Code Execution Successful ===

5. Quick Sort with First Element Pivot

main.py

Run

Share

1- def quick_sort(arr, low, high):

2- if low < high:

3 pivot = arr[low]

4 i = low

5 j = high

6

7- while i < j:

8- while i < high and arr[i] <= pivot:

9 i += 1

10

11- while arr[j] > pivot:

12 j -= 1

13

14- if i < j:

15 arr[i], arr[j] = arr[j], arr[i]

16

17 arr[low], arr[j] = arr[j], arr[low]

18

19 print("After partition:", arr)

20

21 quick_sort(arr, low, j - 1)

22 quick_sort(arr, j + 1, high)

23

24- tests = [

25 [10, 16, 8, 12, 15, 6, 3, 9, 5],

26 [12, 4, 78, 23, 45, 67, 89, 1],

Output

Clear

Input: [10, 16, 8, 12, 15, 6, 3, 9, 5]
After partition: [6, 5, 8, 9, 3, 10, 15, 12, 16]
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]
After partition: [3, 5, 6, 8, 9, 10, 15, 12, 16]
After partition: [3, 5, 6, 8, 9, 10, 12, 15, 16]
Sorted: [3, 5, 6, 8, 9, 10, 12, 15, 16]

Input: [12, 4, 78, 23, 45, 67, 89, 1]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 89, 78]
After partition: [1, 4, 12, 23, 45, 67, 78, 89]
Sorted: [1, 4, 12, 23, 45, 67, 78, 89]

Input: [38, 27, 43, 3, 9, 82, 10]
After partition: [9, 27, 10, 3, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 43, 82]
Sorted: [3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

6. Quick Sort with Middle Element Pivot

main.py

Run

Share

1- def quick_sort(arr, low, high):

2- if low < high:

3 i = low

4 j = high

5 pivot = arr[(low + high) // 2]

6

7- while i <= j:

8- while arr[i] < pivot:

9 i += 1

10

11- while arr[j] > pivot:

12 j -= 1

13

14- if i <= j:

15 arr[i], arr[j] = arr[j], arr[i]

16 i += 1

17 j -= 1

18

19 print("After partition:", arr)

20

21- if low < j:

22 quick_sort(arr, low, j)

23

24- if i < high:

25 quick_sort(arr, i, high)

26

Output

Clear

After partition: [19, 22, 31, 35, 46, 58, 72, 91]
Sorted: [19, 22, 31, 35, 46, 58, 72, 91]

Input: [31, 23, 35, 27, 11, 21, 15, 28]
After partition: [15, 23, 21, 11, 27, 35, 31, 28]
After partition: [15, 11, 21, 23, 27, 35, 31, 28]
After partition: [11, 15, 21, 23, 27, 35, 31, 28]
After partition: [11, 15, 21, 23, 27, 35, 31, 28]
After partition: [11, 15, 21, 23, 27, 28, 31, 35]
After partition: [11, 15, 21, 23, 27, 28, 31, 35]
Sorted: [11, 15, 21, 23, 27, 28, 31, 35]

Input: [22, 34, 25, 36, 43, 67, 52, 13, 65, 17]
After partition: [22, 34, 25, 36, 17, 13, 52, 67, 65, 43]
After partition: [22, 13, 17, 36, 25, 34, 52, 67, 65, 43]
After partition: [13, 22, 17, 36, 25, 34, 52, 67, 65, 43]
After partition: [13, 17, 22, 36, 25, 34, 52, 67, 65, 43]
After partition: [13, 17, 22, 36, 34, 52, 67, 65, 43]
After partition: [13, 17, 22, 25, 34, 36, 52, 67, 65, 43]
After partition: [13, 17, 22, 25, 34, 36, 52, 43, 65, 67]
After partition: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]
After partition: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]
Sorted: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]

=== Code Execution Successful ===

7.Binary Search with Comparison Count

main.py	Run	Output
<pre>1 def binary_search(a, key): 2 low = 0 3 high = len(a) - 1 4 comparisons = 0 5 6 while low <= high: 7 mid = (low + high) // 2 8 comparisons += 1 9 10 if a[mid] == key: 11 return mid, comparisons 12 elif a[mid] < key: 13 low = mid + 1 14 else: 15 high = mid - 1 16 17 return -1, comparisons 18 19 a = [5, 10, 15, 20, 25, 30, 35, 40, 45] 20 key = 20 21 22 index, comparisons = binary_search(a, key) 23 24 print("Index:", index) 25 print("Comparisons:", comparisons)</pre>		Index: 3 Comparisons: 4 === Code Execution Successful ===

8. Binary Search and Mid-Point Analysis

main.py	Run	Output
<pre>1 def binary_search(arr, key): 2 low = 0 3 high = len(arr) - 1 4 5 while low <= high: 6 mid = (low + high) // 2 7 8 print("Low =", low, "High =", high, 9 "Mid =", mid, "Value =", arr[mid]) 10 11 if arr[mid] == key: 12 return mid + 1 13 elif arr[mid] < key: 14 low = mid + 1 15 else: 16 high = mid - 1 17 18 return -1 19 20 21 tests = [22 ([3, 9, 14, 19, 25, 31, 42, 47, 53], 31), 23 ([13, 19, 24, 29, 35, 41, 42], 42), 24 ([20, 40, 60, 80, 100, 120], 60) 25] 26</pre>		Input: [3, 9, 14, 19, 25, 31, 42, 47, 53] Search Key: 31 Low = 0 High = 8 Mid = 4 Value = 25 Low = 5 High = 8 Mid = 6 Value = 42 Low = 5 High = 5 Mid = 5 Value = 31 Position: 6 Input: [13, 19, 24, 29, 35, 41, 42] Search Key: 42 Low = 0 High = 6 Mid = 3 Value = 29 Low = 4 High = 6 Mid = 5 Value = 41 Low = 6 High = 6 Mid = 6 Value = 42 Position: 7 Input: [20, 40, 60, 80, 100, 120] Search Key: 60 Low = 0 High = 5 Mid = 2 Value = 60 Position: 3 === Code Execution Successful ===

9. K Closest Points

main.py	Output
<pre>1 def k_closest(points, k): 2 points.sort(key=lambda p: p[0] * p[0] + p[1] * p[1]) 3 return points[:k] 4 5 6 tests = [7 ([[1, 3], [-2, 2], [5, 8], [0, 1]], 2), 8 ([[1, 3], [-2, 2]], 1), 9 ([[3, 3], [5, -1], [-2, 4]], 2) 10] 11 12 for i, (points, k) in enumerate(tests, 1): 13 result = k_closest(points, k) 14 print("Test Case", i) 15 print("Input:", points) 16 print("K =", k) 17 print("Output:", result) 18 print()</pre>	Test Case 1 Input: [[0, 1], [-2, 2], [1, 3], [5, 8]] K = 2 Output: [[0, 1], [-2, 2]] Test Case 2 Input: [[-2, 2], [1, 3]] K = 1 Output: [[-2, 2]] Test Case 3 Input: [[3, 3], [-2, 4], [5, -1]] K = 2 Output: [[3, 3], [-2, 4]] === Code Execution Successful ===

10. 4-Sum Problem

main.py	Output
<pre>3 def four_sum_count(A, B, C, D): 4 count = Counter() 5 6 for a in A: 7 for b in B: 8 count[a + b] += 1 9 10 answer = 0 11 12 for c in C: 13 for d in D: 14 answer += count[-(c + d)] 15 16 return answer 17 18 19 tests = [20 ([1, 2], [-2, -1], [-1, 2], [0, 2]), 21 ([0], [0], [0], [0]) 22] 23 24 for i, (A, B, C, D) in enumerate(tests, 1): 25 result = four_sum_count(A, B, C, D) 26 27 print("Test Case", i)</pre>	Test Case 1 A = [1, 2] B = [-2, -1] C = [-1, 2] D = [0, 2] Output: 2 Test Case 2 A = [0] B = [0] C = [0] D = [0] Output: 1 === Code Execution Successful ===

11. Median of Medians

main.py

 Share

Run

```
1- def median_of_medians(arr, k):
2-     if len(arr) <= 5:
3-         return sorted(arr)[k - 1]
4-
5-     groups = []
6-
7-     for i in range(0, len(arr), 5):
8-         group = sorted(arr[i:i + 5])
9-         groups.append(group[len(group) // 2])
10
11     pivot = median_of_medians(groups, (len(groups) + 1) // 2)
12
13     smaller = [x for x in arr if x < pivot]
14     equal = [x for x in arr if x == pivot]
15     larger = [x for x in arr if x > pivot]
16
17     if k <= len(smaller):
18         return median_of_medians(smaller, k)
19     elif k <= len(smaller) + len(equal):
20         return pivot
21     else:
22         return median_of_medians(
23             larger,
24             k - len(smaller) - len(equal)
25         )
```

Output

Clear

Test Case 1
Array: [12, 3, 5, 7, 19]
K = 2
K-th Smallest Element: 5

Test Case 2
Array: [12, 3, 5, 7, 4, 19, 26]
K = 3
K-th Smallest Element: 5

Test Case 3
Array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
K = 6
K-th Smallest Element: 6

=== Code Execution Successful ===

12. K-th Smallest Element

main.py	Output
<pre>1 def median_of_medians(arr, k): 2 if len(arr) <= 5: 3 return sorted(arr)[k - 1] 4 5 medians = [] 6 7 for i in range(0, len(arr), 5): 8 group = sorted(arr[i:i + 5]) 9 medians.append(group[len(group) // 2]) 10 11 pivot = median_of_medians(medians, (len(medians) + 1) // 12 2) 13 14 smaller = [x for x in arr if x < pivot] 15 equal = [x for x in arr if x == pivot] 16 larger = [x for x in arr if x > pivot] 17 18 if k <= len(smaller): 19 return median_of_medians(smaller, k) 20 elif k <= len(smaller) + len(equal): 21 return pivot 22 else: 23 return median_of_medians(24 larger, 25 k - len(smaller) - len(equal)</pre>	Test Case 1 Array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] K = 6 K-th Smallest Element: 6 Test Case 2 Array: [23, 17, 31, 44, 55, 21, 20, 18, 19, 27] K = 5 K-th Smallest Element: 21 === Code Execution Successful ===

13.Meet in the Middle – Closest Subset Sum

main.py	Run	Output
<pre> 1 from itertools import combinations 2 from bisect import bisect_left 3 4 def subset_sums(arr): 5 sums = [] 6 7 for r in range(len(arr) + 1): 8 for comb in combinations(arr, r): 9 sums.append((sum(comb), list(comb))) 10 11 return sums 12 13 def meet_in_middle(arr, target): 14 mid = len(arr) // 2 15 16 left = arr[:mid] 17 right = arr[mid:] 18 19 left_sums = subset_sums(left) 20 right_sums = subset_sums(right) 21 22 right_sums.sort() 23 24 best_sum = 0 25 best_subset = [] 26 best_diff = float('inf') </pre>	Run	Test Case 1 Set: [45, 34, 4, 12, 5, 2] Target: 42 Closest Subset: [34, 5, 2] Subset Sum: 41 Difference: 1 Test Case 2 Set: [1, 3, 2, 7, 4, 6] Target: 10 Closest Subset: [4, 6] Subset Sum: 10 Difference: 0 === Code Execution Successful ===

14.Meet in the Middle – Exact Subset Sum

main.py	Run	Output
<pre> 1 from itertools import combinations 2 3 def subset_sums(arr): 4 sums = set() 5 6 for r in range(len(arr) + 1): 7 for comb in combinations(arr, r): 8 sums.add(sum(comb)) 9 10 return sums 11 12 def meet_in_middle(arr, target): 13 mid = len(arr) // 2 14 15 left = arr[:mid] 16 right = arr[mid:] 17 18 left_sums = subset_sums(left) 19 right_sums = subset_sums(right) 20 21 for s in left_sums: 22 if target - s in right_sums: 23 return True 24 25 return False 26 </pre>	Run	Test Case 1 Array: [1, 3, 9, 2, 7, 12] Exact Sum: 15 Output: True Test Case 2 Array: [3, 34, 4, 12, 5, 2] Exact Sum: 15 Output: True === Code Execution Successful ===

15.Strassen's Matrix Multiplication

main.py	Run	Output
<pre>1 def strassen(A, B): 2 a = A[0][0] 3 b = A[0][1] 4 c = A[1][0] 5 d = A[1][1] 6 7 e = B[0][0] 8 f = B[0][1] 9 g = B[1][0] 10 h = B[1][1] 11 12 M1 = (a + d) * (e + h) 13 M2 = (c + d) * e 14 M3 = a * (f - h) 15 M4 = d * (g - e) 16 M5 = (a + b) * h 17 M6 = (c - a) * (e + f) 18 M7 = (b - d) * (g + h) 19 20 C11 = M1 + M4 - M5 + M7 21 C12 = M3 + M5 22 C21 = M2 + M4 23 C22 = M1 - M2 + M3 + M6 24 25 return [[C11, C12], [C21, C22]] 26</pre>		<pre>Matrix A: [1, 7] [3, 5] Matrix B: [6, 8] [4, 2] Product Matrix C: [34, 22] [38, 34] === Code Execution Successful ===</pre>

16.Karatsuba Multiplication

main.py	Run	Output
<pre>1 def karatsuba(x, y): 2 if x < 10 or y < 10: 3 return x * y 4 5 n = max(len(str(x)), len(str(y))) 6 m = n // 2 7 8 p = 10 ** m 9 10 a = x // p 11 b = x % p 12 c = y // p 13 d = y % p 14 15 ac = karatsuba(a, c) 16 bd = karatsuba(b, d) 17 abcd = karatsuba(a + b, c + d) 18 19 ad_bc = abcd - ac - bd 20 21 return ac * p * p + ad_bc * p + bd 22 23 24 x = 1234 25 y = 5678 26</pre>		<pre>X = 1234 Y = 5678 Product = 7006652 === Code Execution Successful ===</pre>